The Foundation
0%

Module 1: QUICK START (...
1.1 What You Need (5 Min Set...
1.2 Your First Folder
1.3 How to Structure Any Pro...

Module 2: WHY THIS WO...
2.1 Video Text Guide (Series ...
2.2 One Line of Python Trigge...
2.3 How a 1953 Word Game E...
2.4 The Ladder That Explains ...
2.5 Clawdbot (Moltbot) Has 1...
2.6 Video as Code: My AI Ani...
2.7 From Nazi Psychology to ...

Module 3: FOLDER ARCHI...
3.1 The Full Walkthrough (23 ...
3.2 Customizing for Your Use ...
3.3 Common Mistakes and H...

2.5 Clawdbot (Moltbot) Has 100K Stars

13:16

What You'll Get From This

You will understand the difference between orchestration and intelligence, why most production AI systems are 90% traditional code, and how to look at any AI tool and figure out where the actual value lives.

The written version below covers the same ground as the video. Watch the video above, read the companion, or both. The written version is designed to be something you can hand to Claude or reference later.

The Lesson

The hype is real

Here is the pitch. One assistant, everywhere you already are. Message it on WhatsApp, get a response on Telegram, ask it to check your calendar from Slack, and it just works. Access your files, your browser, your tools, all from one place.

The enthusiasm is real. But here is what most people are missing.

If you open the codebase, all 50,000 lines of it, read every file, check every folder, you will not find a model. No weights. No training data. There is no AI inside of it. None.

So what is everyone excited about?

What is orchestration?

To understand what Moltbot actually is, we need to talk about orchestration.

Think about an actual orchestra. Dozens of musicians. Brass, strings, woodwinds, percussion. Each one a specialist. Each one capable of making music on their own. But the conductor does not play an instrument. The conductor does not make any sound at all. The conductor's job is to coordinate who plays when, how loud, in what order, when to speed up, when to slow down, when to pause.

The musicians are the talent. The conductor is the organizer.

Now replace musicians with AI models, tools, and messaging platforms. That is orchestration.

The shapes: how routing works

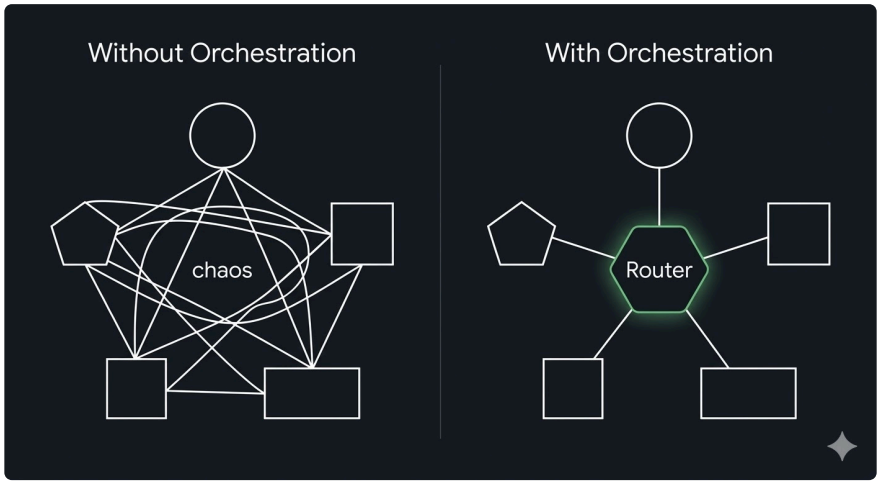
Here is the simplest way to picture this.

A circle is your message: "What is on my calendar today?" A pentagon is an AI model: Claude, GPT, Llama. A square is a tool: your calendar, your browser, your file system. A rectangle is a channel: WhatsApp, Slack, Discord.

Without an orchestration layer, here is what happens. Your message has to find the AI. The AI has to figure out what tool it needs. The tool has to be called. The response has to get back to the AI. Then the whole thing has to find its way back to

Now add an orchestration layer in the middle. A central hub that speaks every language. Your message goes in. The orchestrator routes it to the right AI. The AI says "I need calendar data." The orchestrator calls the calendar tool, gets the response, sends it back to the AI. The AI formulates an answer. The orchestrator sends it back to WhatsApp or Slack or wherever you started.

Same task. Clean architecture. Everything flows through one intelligent router.



Getting the terms straight

People use these words interchangeably. They should not.

AI Model — The actual intelligence. GPT, Claude, Llama. The thing that processes language and generates responses. Moltbot is not this.

Wrapper — A thin layer around an API. Minimal added value. Takes someone else's AI and puts a slightly different interface on it. Moltbot is not really this either.

Agent — An AI that can take actions on its own. Moltbot enables this, but it needs an external AI to do it.

Orchestration layer — Coordinates AI plus tools plus channels plus context. This is what Moltbot actually is.

deliveries where they need to go.

Inside the architecture

So what does the codebase actually look like if there is no AI model inside?

Moltbot is built in layers. At the top, a channel layer that connects to messaging platforms: WhatsApp (using a library called Baileys), Telegram (through Grammy), Discord (through Discord.js), Signal (through Signal CLI). Each of these is a solid, maintained open-source library that knows how to speak the platform's language.

In the middle, a gateway. This is the brain of the operation, and not an AI brain. A routing brain. It is a websocket server running locally on your machine. Every message from every channel flows through here. The gateway decides where it goes. It manages sessions so the AI remembers your conversation context. It handles authentication so random people cannot message your AI.

At the bottom, two things: tools and AI models. The tools are built in (browser automation, bash commands, file operations, scheduling, webhooks). The AI models are empty hooks. You bring your own. You plug in your Anthropic API key or your OpenAI key or you run a local model through Llama and point it at Moltbot. Moltbot handles all the API calls. The intelligence, the actual thinking, comes from somewhere else. Somewhere you are probably already paying for.

The 60/30/10 breakdown

Here is where this gets practical. The Moltbot codebase breaks down roughly like this:

- **~35%** is channel integrations. The code that talks to WhatsApp, Telegram, Discord, and the rest.
- **~25%** is the gateway. Routing logic, session management, security.

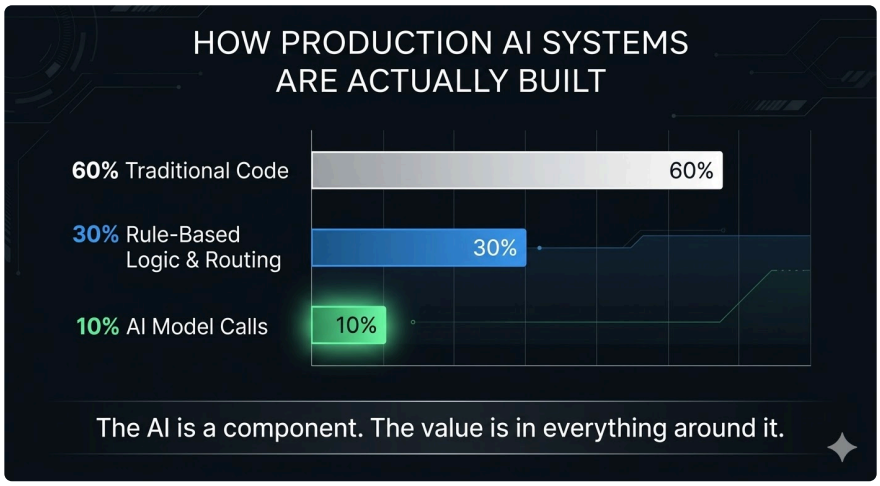
- **~10%** is CLI configuration and AI calls. Just the code that sends messages to whatever model you configured.

This maps almost perfectly to a framework I use in all of my workshops and consulting work: **60/30/10**.

Build AI tools with roughly 60% traditional code (platform integrations, networking, file handling, stuff that has nothing to do with AI), 30% rule-based logic (routing decisions, management, security rules, smart code that is still not AI models), and only about 10% actual AI processing

In Moltbot's case, that 10% is even pushed externally. It is API calls to someone else's AI.

This is not a criticism. This is a recommendation on architecture. Most production AI systems look like this. The AI is a component, not the whole thing. The value is in everything around the AI, not the AI itself. Moltbot just makes this more visible than most tools do.



The question nobody is asking

Here is where we need to be honest. There is a question hanging over Moltbot that nobody seems to be asking.

What happens when the big players build this themselves?

hosting. Just Claude, everywhere.

Why would anyone need Moltbot?

This happens constantly in tech. A platform adds a feature that makes your whole product unnecessary. Some people call it getting "Sherlocked" (after Apple's habit of building features that killed third-party apps).

But there is another way this plays out. Moltbot is model agnostic. You can swap Claude for GPT or Llama or a local model. No vendor lock-in. Moltbot is self-hosted. Your conversations stay on your hardware. Your data, your rules. And Moltbot is open source. 200 contributors. Community-driven development. Features that matter to users, not shareholders.

So the question becomes: in a world of orchestration layers and frontier models, where does the value actually live?

What this teaches you

Forget Moltbot for a second. Here is the lesson underneath it.

Every time you look at an AI tool, an AI startup, an AI framework, ask yourself three questions:

1. **Where is the intelligence?** Is it in the product, or is the product calling someone else's AI? If you removed the AI model, what would be left?

2. **Where is the value?** Is the value in the AI itself, or in the routing, the integrations, the architecture around it? Moltbot has zero AI and 100,000 stars. The value is the orchestration.

3. **What gets commoditized next?** If Anthropic or OpenAI builds the orchestration layer natively, what happens to the product? If the AI models get cheaper, what happens to the business model? Value lives above whatever just got commoditized. Always.

you building at the layer that is about to become free?

This is also why the folder architecture from **Section 1.2** works the way it does. You are not building a custom AI. You are building an orchestration layer in plain English. Folders, context files, and routing tables. The intelligence comes from Claude (or whatever model you use). Your job is the routing, the structure, the architecture around it. That is where the lasting value is.

Resources

- **Moltbot GitHub**

Go Deeper

- **The 60/30/10 framework** gets a full course in **Premium: The Vault**. Free tier gives you the concept. Premium teaches you how to apply it to your own builds and client work.
- This video looks at the stack from the business and architecture side. **Section 2.2** looks at it from the technical side. Same conclusion: value lives above whatever just got commoditized.
- **Section 2.4** expands on the "what gets commoditized next" question and applies it to companies that spent six figures building at the wrong layer.
- The folder architecture from **Section 1.2** is a personal orchestration layer. You are the conductor. Claude is the musician. The folders and context files are the score.

⏭️ Next Up: 2.6 Video as Code: My AI Animation Stack

You just saw how a 100K-star project is 90% orchestration and 10% AI calls. Next: how one person replaces an entire production team using the same principle. Folders, routing, and

Matthew Creamer

24d • Lesson Check-Ins

Foundations 2.5 Check-In

This one breaks down orchestration. Vote below, then pick one AI tool you actually use and tell us in the comments:

Poll 140 members have voted

41 44

New comment 5h ago